

Supply Chain X-Ray

Complete Pipeline Logic — Every Step, Every API Call

From company name input to finished BOM-filtered graph with data provenance

DATA STRUCTURES

Everything in the system flows through two arrays. Every step below adds to them.

```
// These two arrays ARE your graph. Nothing else needed. const nodes = []; // { id, name, country, tier, hsn, source, confidence } const edges = []; // { from, to, hsn, commodity } const cache = {};  
// key: "companyName:hsn" → value: API response // Source labels: // "cbp-live" → ImportYeti  
forward search (Tier 1) // "cbp-reverse" → ImportYeti reverse search (Tier 2) // "sec+comtrade"  
→ SEC EDGAR name + Comtrade country match (Tier 3+) // "sec-edgar" → SEC EDGAR name, no  
Comtrade match (Tier 3+) // "comtrade+wiki" → Comtrade country + Wikidata company (Tier 3+) //  
"seed" → Pre-loaded reference data
```

STEP 1 — USER ENTERS COMPANY NAME

Input: "General Motors"

Query ImportYeti for all inbound shipments where GM is the consignee.

Raw ImportYeti response (sample)

```
ImportYeti search: consignee = "GENERAL MOTORS" Returns ~hundreds of shipment records: { shipper:  
"GM KOREA COMPANY", hsn: "8703.23", country: "South Korea", shipments: 842 } { shipper: "GM  
KOREA COMPANY", hsn: "8703.22", country: "South Korea", shipments: 156 } { shipper: "CAMI  
AUTOMOTIVE INC", hsn: "8703.23", country: "Canada", shipments: 634 } { shipper: "YAZAKI  
CORPORATION", hsn: "8544.30", country: "Japan", shipments: 211 } { shipper: "MAGNA  
INTERNATIONAL", hsn: "8708.99", country: "Canada", shipments: 189 } { shipper: "AISIN  
CORPORATION", hsn: "8708.40", country: "Japan", shipments: 97 } { shipper: "NSK LTD",  
hsn: "8482.10", country: "Japan", shipments: 63 } { shipper: "BRIDGESTONE CORP", hsn:  
"4011.10", country: "Japan", shipments: 55 } { shipper: "OFFICE DEPOT INC", hsn:  
"4820.10", country: "China", shipments: 12 } { shipper: "STAPLES INC", hsn:  
"9403.20", country: "USA", shipments: 8 } ...
```

Extract unique HSN codes → look up descriptions

```
// Deduplicate HSN codes from all records unique HSN codes found: 8703.22 → look up USITC →  
"Passenger vehicles, spark-ignition, 1000-1500cc" 8703.23 → look up USITC → "Passenger vehicles,  
spark-ignition, 1500-3000cc" 8544.30 → look up USITC → "Wiring harness sets for vehicles" 8708.99  
→ look up USITC → "Vehicle parts, not elsewhere classified" 8708.40 → look up USITC → "Gearboxes and  
parts thereof" 8482.10 → look up USITC → "Ball or roller bearings" 4011.10 → look up USITC →  
"Pneumatic tyres for passenger vehicles" 4820.10 → look up USITC → "Registers, notebooks (paper)"  
9403.20 → look up USITC → "Metal furniture" // Present this list as dropdown to user // User selects:  
8703.23
```

Tier-0 node created:

```
nodes.push({ id: "general-motors", name: "General Motors Company", country: "USA", tier: 0,  
hsn: "8703.23", source: "user-input", confidence: "anchor" })
```

STEP 2 — BUILD TIER-1 WITH BOM FILTER

BOM filter #1: filter against parent HSN 8703.23

```
// Load BOM tree bomTree["8703.23"] =  
["8708","8407","8544","8482","4011","7009","7225","7208","8501","8507"] // For each ImportYeti  
record, check if shipper's HSN matches any valid input for (const record of importYetiRecords) {  
  const childPrefix = record.hsn.replace('.', '').substring(0, 4);  const isValid =  
  bomTree["8703.23"].some(v => v.replace('.', '').substring(0,4) === childPrefix);  if (isValid) {  
    // KEEP — add as Tier-1 node + edge  } else {    // PRUNE — not a manufacturing input to a passenger  
    vehicle  } }
```

What gets KEPT (green = valid BOM input)

```
KEPT — valid BOM inputs to 8703.23 (passenger vehicle):  GM Korea      → 8703.23 → prefix 8703 →  
matches? YES (sub-assembly of same product)  CAMI Auto      → 8703.23 → prefix 8703 → YES  Yazaki  
→ 8544.30 → prefix 8544 → matches "8544" in bomTree → YES (wiring)  Magna      → 8708.99 →  
prefix 8708 → matches "8708" → YES (body parts)  Aisin      → 8708.40 → prefix 8708 → matches  
"8708" → YES (gearbox)  NSK      → 8482.10 → prefix 8482 → matches "8482" → YES (bearings)  
Bridgestone → 4011.10 → prefix 4011 → matches "4011" → YES (tyres)
```

What gets PRUNED (red = not a vehicle input)

```
PRUNED — not in bomTree["8703.23"]:  Office Depot → 4820.10 → prefix 4820 → NOT in list → PRUNED  
(paper notebooks)  Staples → 9403.20 → prefix 9403 → NOT in list → PRUNED (metal furniture)
```

Resulting nodes and edges added

```
// Tier-1 nodes nodes.push({ id: "gm-korea",    name: "GM Korea Company",    country: "South Korea",  
tier: 1, hsn: "8703.23", source: "cbp-live" }) nodes.push({ id: "cami",                name: "CAMI  
Automotive",    country: "Canada",    tier: 1, hsn: "8703.23", source: "cbp-live" }) nodes.push({  
id: "yazaki",    name: "Yazaki Corporation",    country: "Japan",    tier: 1, hsn: "8544.30",  
source: "cbp-live" }) nodes.push({ id: "magna",    name: "Magna International",    country:  
"Canada",    tier: 1, hsn: "8708.99", source: "cbp-live" }) nodes.push({ id: "aisin",    name:  
"Aisin Corporation",    country: "Japan",    tier: 1, hsn: "8708.40", source: "cbp-live" })  
nodes.push({ id: "nsk",    name: "NSK Ltd",    country: "Japan",    tier: 1, hsn:  
"8482.10", source: "cbp-live" }) nodes.push({ id: "bridgestone",    name: "Bridgestone Corp",  
country: "Japan",    tier: 1, hsn: "4011.10", source: "cbp-live" }) // Tier-1 edges edges.push({  
from: "general-motors", to: "gm-korea",    hsn: "8703.23", commodity: "Passenger vehicles" })  
edges.push({ from: "general-motors", to: "cami",    hsn: "8703.23", commodity: "Passenger  
vehicles" }) edges.push({ from: "general-motors", to: "yazaki",    hsn: "8544.30", commodity:  
"Wiring harness sets" }) edges.push({ from: "general-motors", to: "magna",    hsn: "8708.99",  
commodity: "Vehicle body parts" }) edges.push({ from: "general-motors", to: "aisin",    hsn:  
"8708.40", commodity: "Gearbox assemblies" }) edges.push({ from: "general-motors", to: "nsk",  
hsn: "8482.10", commodity: "Ball bearings" }) edges.push({ from: "general-motors", to: "bridgestone",  
hsn: "4011.10", commodity: "Pneumatic tyres" })
```

STEP 3 — BUILD TIER-2 VIA REVERSE IMPORTYETI

Critical: BOM parent changes per node

Each Tier-1 node was connected by a specific HSN code. That connecting code becomes the BOM parent for filtering the next tier. You do NOT filter against the original 8703.23 — you filter against the edge that brought you here.

Yazaki (connected via 8544.30 — wiring harnesses)

```
// Reverse search: ImportYeti consignee = "Yazaki Corporation" // Returns Yazaki's own inbound
shipments: { shipper: "FURUKAWA ELECTRIC", hsn: "7408.11", country: "Japan" } // copper wire {
shipper: "SUMITOMO ELECTRIC", hsn: "7408.11", country: "Japan" } // copper wire { shipper:
"KUREHA CORPORATION", hsn: "3926.90", country: "Japan" } // plastic connectors { shipper: "DAI
NIPPON PRINTING", hsn: "4911.10", country: "Japan" } // trade catalogues { shipper: "TOKYO OFFICE
SUPPLY", hsn: "4820.10", country: "Japan" } // stationery // BOM filter with parent = 8544.30
(wiring harness) bomTree["8544.30"] = ["7408","3926","7606","4016"]
```

```
KEPT: Furukawa Electric → 7408.11 → matches "7408" → YES (copper wire → wiring harness) Sumitomo
Electric → 7408.11 → matches "7408" → YES Kureha Corp → 3926.90 → matches "3926" → YES
(plastic connectors)
```

```
PRUNED: Dai Nippon Printing → 4911.10 → NOT in bomTree["8544.30"] → PRUNED (catalogues) Tokyo
Office Supply → 4820.10 → NOT in list → PRUNED (stationery)
```

GM Korea (connected via 8703.23 — vehicles)

```
// Reverse search: ImportYeti consignee = "GM Korea" { shipper: "POSCO", hsn: "7225.19",
country: "South Korea" } // electrical steel { shipper: "HYUNDAI STEEL", hsn: "7208.51",
country: "South Korea" } // hot-rolled steel { shipper: "MANDO CORP", hsn: "8708.30",
country: "South Korea" } // brake systems { shipper: "KUMHO TIRE", hsn: "4011.10", country:
"South Korea" } // tyres { shipper: "SAMSUNG DISPLAY", hsn: "9013.80", country: "South Korea" }
// LCD displays { shipper: "CJ LOGISTICS", hsn: "4819.10", country: "South Korea" } //
cardboard boxes // BOM filter with parent = 8703.23 bomTree["8703.23"] =
["8708","8407","8544","8482","4011","7009","7225","7208","8501","8507"]
```

```
KEPT: POSCO → 7225.19 → matches "7225" → YES (steel for stamped body parts) Hyundai
Steel → 7208.51 → matches "7208" → YES (hot-rolled steel for chassis) Mando Corp → 8708.30 →
matches "8708" → YES (brake assemblies) Kumho Tire → 4011.10 → matches "4011" → YES (tyres)
```

```
PRUNED: Samsung Display → 9013.80 → NOT in list → PRUNED (display is accessory, not core BOM) CJ
Logistics → 4819.10 → NOT in list → PRUNED (packaging cardboard)
```

```
// Tier-2 nodes added nodes.push({ id: "furukawa", name: "Furukawa Electric", country: "Japan",
tier: 2, hsn: "7408.11", source: "cbp-reverse" }) nodes.push({ id: "sumitomo", name: "Sumitomo
Electric", country: "Japan", tier: 2, hsn: "7408.11", source: "cbp-reverse" }) nodes.push({ id:
"posco", name: "POSCO", country: "South Korea", tier: 2, hsn: "7225.19", source: "cbp-
reverse" }) nodes.push({ id: "hyundai-s", name: "Hyundai Steel", country: "South Korea", tier: 2,
hsn: "7208.51", source: "cbp-reverse" }) nodes.push({ id: "mando", name: "Mando Corp",
country: "South Korea", tier: 2, hsn: "8708.30", source: "cbp-reverse" }) nodes.push({ id: "kumho",
name: "Kumho Tire", country: "South Korea", tier: 2, hsn: "4011.10", source: "cbp-reverse" })
nodes.push({ id: "kureha", name: "Kureha Corp", country: "Japan", tier: 2, hsn:
"3926.90", source: "cbp-reverse" }) // Tier-2 edges edges.push({ from: "yazaki", to: "furukawa",
hsn: "7408.11", commodity: "Copper winding wire" }) edges.push({ from: "yazaki", to: "sumitomo",
hsn: "7408.11", commodity: "Copper winding wire" }) edges.push({ from: "yazaki", to: "kureha",
hsn: "3926.90", commodity: "Plastic connectors" }) edges.push({ from: "gm-korea", to: "posco",
hsn: "7225.19", commodity: "Silicon-electrical steel" }) edges.push({ from: "gm-korea", to: "hyundai-
s", hsn: "7208.51", commodity: "Hot-rolled flat steel" }) edges.push({ from: "gm-korea", to: "mando",
hsn: "8708.30", commodity: "Brake systems" }) edges.push({ from: "gm-korea", to: "kumho", hsn:
"4011.10", commodity: "Pneumatic tyres" })
```

STEP 4 — BUILD TIER-3 VIA COMTRADE + SEC EDGAR

ImportYeti can't go further. Switch to Comtrade + SEC EDGAR.

Take each Tier-2 node. The BOM tree tells you what they need as inputs. Query Comtrade for those inputs by country. Then find the company name via SEC EDGAR.

POSCO (connected via 7225.19 — electrical steel)

```
// What does POSCO need to make electrical steel? bomTree["7225.19"] = ["2601","7201","7202","7203"]
// → iron ore (2601), pig iron (7201), ferro-alloys (7202/7203) // API Call 1: UN Comtrade GET
https://comtradeapi.un.org/data/v1/get/C/A/HS ?reporterCode=410 // South Korea
&flowCode=M // Import &cmdCode=260111 // Iron ore (first valid input)
&period=2023 &partnerCode=0 // All partners Response: Australia → $8.2B (58%)
Brazil → $3.9B (27%) South Africa → $1.1B (8%) Canada → $0.4B (3%)
```

```
// API Call 2: SEC EDGAR GET https://efits.sec.gov/LATEST/search-index?
q="iron+ore+supplier"&entity=POSCO&forms=20-F Found in POSCO 20-F, page 42: "We procure iron ore
primarily from Vale S.A. (Brazil), Rio Tinto (Australia), and BHP Group (Australia)."
```

Cross-reference: SEC names vs Comtrade countries

```
Cross-reference result: Vale S.A. → Brazil → Comtrade says Brazil = 27% → MATCH →
"sec+comtrade" [verified] Rio Tinto → Australia → Comtrade says Australia = 58% → MATCH →
"sec+comtrade" [verified] BHP Group → Australia → Comtrade says Australia = 58% → MATCH →
"sec+comtrade" [verified] BOM filter: all three supply 2601.11 (iron ore) bomTree["7225.19"]
includes "2601" → all three KEPT
```

```
// Tier-3 nodes from POSCO branch nodes.push({ id: "vale", name: "Vale S.A.", country:
"Brazil", tier: 3, hsn: "2601.11", source: "sec+comtrade", confidence: "verified" }) nodes.push({
id: "rio-tinto", name: "Rio Tinto", country: "Australia", tier: 3, hsn: "2601.11", source:
"sec+comtrade", confidence: "verified" }) nodes.push({ id: "bhp", name: "BHP Group", country:
"Australia", tier: 3, hsn: "2601.11", source: "sec+comtrade", confidence: "verified" }) edges.push({
from: "posco", to: "vale", hsn: "2601.11", commodity: "Iron ore (non-agglomerated)" })
edges.push({ from: "posco", to: "rio-tinto", hsn: "2601.11", commodity: "Iron ore (Pilbara)" })
edges.push({ from: "posco", to: "bhp", hsn: "2601.11", commodity: "Iron ore" })
```

Furukawa Electric (connected via 7408.11 — copper wire)

```
// What does Furukawa need to make copper wire? bomTree["7408.11"] = ["7403","7407"] // → refined
copper cathodes (7403), copper rod (7407) // Comtrade: who exports copper cathodes (7403.11) to
Japan? Reporter = Japan (392), HS = 740311, Flow = Import Response: Chile → $2.1B (52%)
Australia → $1.1B (28%) Indonesia → $0.5B (12%) // SEC EDGAR: search Furukawa 20-F for
"copper" Found: "Copper rod procured from Mitsubishi Materials Corp (Japan) and refined cathodes
from Aurubis AG (Germany)." Cross-reference: Mitsubishi Materials → Japan → domestic (not in
Comtrade imports) → source: "sec-edgar" Aurubis AG → Germany → not in Comtrade top 3 but
legitimate → source: "sec-edgar" // Wikidata fallback for Chile (Comtrade #1, 52%): // SPARQL: copper
mining + Chile → Codelco, Antofagasta plc Codelco → Chile → matches Comtrade #1 → source:
"comtrade+wiki" [verified]
```

```
// Tier-3 nodes from Furukawa branch nodes.push({ id: "aurubis", name: "Aurubis AG",
country: "Germany", tier: 3, hsn: "7403.11", source: "sec-edgar" }) nodes.push({ id: "mitsubishi-m",
name: "Mitsubishi Materials", country: "Japan", tier: 3, hsn: "7403.11", source: "sec-edgar" })
nodes.push({ id: "codelco", name: "Codelco", country: "Chile", tier: 3, hsn:
"7403.11", source: "comtrade+wiki", confidence: "verified" }) edges.push({ from: "furukawa", to:
"aurubis", hsn: "7403.11", commodity: "Refined copper cathodes" }) edges.push({ from: "furukawa",
to: "mitsubishi-m", hsn: "7403.11", commodity: "Copper rod" }) edges.push({ from: "furukawa", to:
"codelco", hsn: "7403.11", commodity: "Refined copper cathodes" })
```

STEP 5 — TIER-4 (SAME METHOD, BOM CASCADES DOWN)

Aurubis AG (connected via 7403.11 — refined copper cathodes)

```
// What does Aurubis need to make copper cathodes? bomTree["7403.11"] = ["2603"] // → copper ore and concentrates (2603) // Comtrade: who exports copper ore (2603.00) to Germany? Reporter = Germany (276), HS = 260300, Flow = Import Response: Chile → $1.8B (44%) Peru → $0.9B (22%) Brazil → $0.6B (15%) // SEC EDGAR: Aurubis annual report "Copper concentrates sourced from Codelco (Chile), Antofagasta plc (Chile), and Southern Copper (Peru)." Cross-reference: Codelco → Chile → Comtrade #1 → "sec+comtrade" [verified] Antofagasta plc → Chile → Comtrade #1 → "sec+comtrade" [verified] Southern Copper → Peru → Comtrade #2 → "sec+comtrade" [verified]
```

Cache reuse kicks in

```
// Codelco already exists in nodes[] from Step 4! // Check cache: cache["codelco:7403.11"] exists // → REUSE existing node, just add a new edge // Only Antofagasta and Southern Copper are new nodes.push({ id: "antofagasta", name: "Antofagasta plc", country: "Chile", tier: 4, hsn: "2603.00", source: "sec+comtrade" }) nodes.push({ id: "southern-cu", name: "Southern Copper", country: "Peru", tier: 4, hsn: "2603.00", source: "sec+comtrade" }) edges.push({ from: "aurubis", to: "codelco", hsn: "2603.00", commodity: "Copper ore concentrates" }) edges.push({ from: "aurubis", to: "antofagasta", hsn: "2603.00", commodity: "Copper ore concentrates" }) edges.push({ from: "aurubis", to: "southern-cu", hsn: "2603.00", commodity: "Copper ore concentrates" })
```

Terminal check

```
// Check bomTree for 2603.00: bomTree["2603.00"] = [] // empty array = raw ore = TERMINAL NODE // Codelco, Antofagasta, Southern Copper are the last resolvable tier. // Traversal stops here on this branch.
```

STEP 6 — THE RECURSION ENGINE (COMPLETE CODE)

```
const bomTree = require('./bomTree.json'); async function traverseTier(parentNode, parentEdgeHSN, currentTier, maxTier = 6) { // Stop conditions if (currentTier > maxTier) return; const validInputs = bomTree[parentEdgeHSN]; if (!validInputs || validInputs.length === 0) return; // terminal node let supplierEdges = []; // — Tier 1-2: ImportYeti ————— if (currentTier <= 2) { const cacheKey = `importyeti:${parentNode.name}`; if (cache[cacheKey]) { supplierEdges = cache[cacheKey]; } else { supplierEdges = await fetchImportYeti(parentNode.name); // your IV fetch function cache[cacheKey] = supplierEdges; } } // — Tier 3+: Comtrade + SEC EDGAR + Wikidata ————— else { for (const inputHSN of validInputs) { // Step A: Comtrade → get countries const countries = await fetchComtrade(parentNode.country, inputHSN); // Step B: SEC EDGAR → get company names const secSuppliers = await searchSECEdgar(parentNode.name, inputHSN); // Step C: Cross-reference for (const sec of secSuppliers) { const matchesComtrade = countries.some(c => c.name === sec.country); supplierEdges.push({ hsn: inputHSN, name: sec.name, country: sec.country, source: matchesComtrade ? 'sec+comtrade' : 'sec-edgar', confidence: matchesComtrade ? 'verified' : 'high' }); } // Step D: Wikidata fallback for unmatched Comtrade countries const secCountries = secSuppliers.map(s => s.country); for (const ct of countries.slice(0, 3)) { if (!secCountries.includes(ct.name)) { const wikiCompanies = await queryWikidata(ct.name, inputHSN); for (const wc of wikiCompanies) { supplierEdges.push({ hsn: inputHSN, name: wc.name, country: ct.name, source: 'comtrade+wiki', confidence: 'medium' }); } } } // — BOM filter ————— const filtered = supplierEdges.filter(edge => { const childPrefix = edge.hsn.replace('.', '').substring(0, 4); return validInputs.some(v => v.replace('.', '').substring(0, 4) === childPrefix); }); // — Add to graph ————— for (const edge of filtered) { const nodeId =
```

```
edge.name.toLowerCase().replace(/^[a-z0-9]/g, '-'); // Cache check – reuse existing node if
(!nodes.find(n => n.id === nodeId)) { nodes.push({ id: nodeId, name: edge.name,
country: edge.country, tier: currentTier, hsn: edge.hsn, source: edge.source
|| (currentTier <= 2 ? 'cbp-reverse' : edge.source), confidence: edge.confidence });
} edges.push({ from: parentNode.id, to: nodeId, hsn: edge.hsn, commodity:
getHSDescription(edge.hsn) }); // — RECURSE with new parent HSN ————— const
childNode = nodes.find(n => n.id === nodeId); await traverseTier(childNode, edge.hsn, currentTier
+ 1, maxTier); } } // — Start traversal ————— const gmNode = { id:
'general-motors', name: 'General Motors', country: 'USA' }; await traverseTier(gmNode, '8703.23', 1);
```

FINAL GRAPH OUTPUT — GM (8703.23)

This is the complete graph the pipeline produces:

```
GM (8703.23) — Tier 0 [user-input] | | GM Korea (8703.23) ————— Tier 1 [cbp-live] |
| POSCO (7225.19) ————— Tier 2 [cbp-reverse] | | Vale S.A. (2601.11) —————
Tier 3 [sec+comtrade] VERIFIED | | Rio Tinto (2601.11) ————— Tier 3 [sec+comtrade] VERIFIED
| | BHP Group (2601.11) ————— Tier 3 [sec+comtrade] VERIFIED | | (terminal -
raw ore, bomTree = []) | | Hyundai Steel (7208.51) ————— Tier 2 [cbp-reverse] | | Vale
S.A. (2601.11) ————— (reused from cache) | | Mando Corp (8708.30) ————— Tier 2 [cbp-
reverse] | | Kumho Tire (4011.10) ————— Tier 2 [cbp-reverse] | | CAMI Automotive (8703.23)
————— Tier 1 [cbp-live] | | Yazaki Corp (8544.30) ————— Tier 1 [cbp-live] | |
Furukawa Electric (7408.11) ————— Tier 2 [cbp-reverse] | | Aurubis AG (7403.11) ————— Tier 3
[sec-edgar] | | Codelco (2603.00) ————— Tier 4 [sec+comtrade] VERIFIED | | |
Antofagasta (2603.00) — Tier 4 [sec+comtrade] VERIFIED | | | Southern Copper (2603) — Tier
4 [sec+comtrade] VERIFIED | | | (terminal - raw ore) | | Mitsubishi Mat.
(7403.11) — Tier 3 [sec-edgar] | | Codelco (7403.11) ————— Tier 3 [comtrade+wiki]
(reused) | | Sumitomo Electric (7408.11) ————— Tier 2 [cbp-reverse] | | Kureha Corp (3926.90)
————— Tier 2 [cbp-reverse] | | Magna International (8708.99) ————— Tier 1 [cbp-live] | | Aisin
Corporation (8708.40) ————— Tier 1 [cbp-live] | | NSK Ltd (8482.10) ————— Tier 1 [cbp-
live] | | Bridgestone Corp (4011.10) ————— Tier 1 [cbp-live] Total: 20 nodes, 22 edges Reused
from cache: 2 nodes (Vale, Codelco) Pruned by BOM filter: 4 records (Office Depot, Staples, Samsung
Display, CJ Logistics, etc.)
```

Every node has: name, country, tier, HSN code, data source label, confidence level. Every edge has: HSN code + commodity description. The graph is complete, BOM-filtered, and traceable from T0 to raw ore.

API SUMMARY — WHAT GETS CALLED WHERE

Tier	API	Input	Output	Cost
T0	USITC HTS	HSN code string	Commodity description	Free, no key, no login
T1	ImportYeti	Company name as consignee	Shipper list + HSN	Free with login, 25 pages/IP
T2	ImportYeti	Tier-1 company as consignee	Their shippers + HSN	Same as above (reverse)
T3	UN Comtrade	Reporter country + HS code	Partner countries	Free key, 100 calls/day
T3	SEC EDGAR	Company name + commodity keyword	Supplier names	Free, no key, no login

T3	Wikidata	Country + industry Wikidata ID	Company names	Free, no key, no limit
T4+	Same as T3	Recursive — same three sources	Same structure	Cached — fewer new calls